

3. Working with tries (Dry and Wet)

Given a Trie that is designed using TrieNode, nodes that contains an array of 26 child nodes (see tutorial 8).

1. Write the pseudocode of a function that takes a Trie and converts it into a Patricia Trie.

```
java
1 toPatriciaTrie(node)
2   //Base case: if node is null, return
3   if node is null then return
4   //recurse all child
5   for each child in node.children do
6     toPatriciaTrie(child)
7   end for
8   for each child in node.children do
9     if child != null do
10      if |child.children| = 1 and !child.EndOfWord then
11        grandChild = child.child
12        child.index = grandChild.index
13        child.children = grandChild.children
14        grandChild is a word iff child is a word
15        for each node in child.children do
16          if (nod != null) then nod.parent = child end if
17        end for
18      end if
19    end for
```

2. Write the pseudocode of search, insert and remove in a Patricia Trie.

Here is the structure of Patricia Trie Node

- search

```
java
1 //find node according to the prefix
2 findNode(prefix)
3   if isValidWord(prefix) is false then throw exception end if
4   current = root
5   while current != null and current.index < |prefix| do
6     i = current.index
7     ch = prefix.CharAt(i)
8     child = current.getChild(ch)
9     if child == null return null end if
10    current = child
11  end while
12  return current
13
14 search(word)
15   if isValidWord(prefix) is false then throw exception end if
16   node = findNode(word)
17   //this node is a word iff it's not null, it is a word and equals word
18   return node != null and node.isEndOfWord and node.fullWord.equals(word)
```

- insert

```
java
1 insert(word)
2   if isValidWord(word) is false then throw exception end if
3   if search(word) is true then return end if
4   current = root
5   while current.index < word.length do
6     ch = word.charAt(current.index)
7     child = current.getChild(ch)
8     if child is null then
9       insertNode(current, ch, word)
```

```

10         return
11     else
12         if prepareSplitting(current, child, word) is true then
13             return
14         end if
15         current = child
16     end if
17 end while

```

These functions are used to insert

```

java
1 //return if we need to process the child depends on whether we need to split the path
2 prepareSplitting(parent, child, word)
3     existingWord = getOneWord(child)
4     splitNodeIndex = findSplitedNodeIndex(parent.index, child.index, word, existingWord)
5     // if we find the splited index, the splitNodeIndex is not -1
6     // But if splitNodeIndex == -1, it is also possible that the word is some prefix of existingWord
7     if splitNodeIndex != -1 or word.length < child.index then
8         if splitNodeIndex != -1 then
9             // find splitIndex, also considering the case that the word is some prefix of existingWord
10            splitIndex = splitNodeIndex
11        else
12            splitIndex = word.length
13        end if
14        split(parent, child, word, existingWord, splitIndex)
15        return true
16    end if
17    return false
18
19 findSplitedNodeIndex(parentIndex, childIndex, word, existingWord)
20     length = min(word.length, childIndex)
21     i = parentIndex + 1
22     while i < length do
23         if existingWord.charAt(i) != word.charAt(i) then
24             return i
25         end if
26         i++
27     end while
28     //no need to split
29     return -1
30
31 //split in splitIndex, word is at splitIndex. And move the child to splitIndexNode
32 split(parent, child, word, existingWord, splitIndex)
33     ch = existingWord.charAt(parent.index)
34     //create the splitNode
35     splitNode = new PatriciaTrieNode()
36     splitNode.index = splitIndex
37     parent.setChild(ch, splitNode)
38
39     // move the child to the splitNode
40     oldChildChar = existingWord.charAt(splitIndex)
41     splitNode.setChild(oldChildChar, child)
42
43     //add the word in splitNode
44     if splitIndex == word.length then
45         //do not need to create branch
46         splitNode.isEndOfWord = true
47         splitNode.fullWord = word
48     else
49         //need to branch
50         newNode = new PatriciaTrieNode()
51         newNode.index = word.length
52         newNode.isEndOfWord = true

```

```
53     newNode.fullWord = word
54     newChildChar = word.charAt(splitIndex)
55     splitNode.setChild(newChildChar, newNode)
56 end if
```

- remove

```
java
1 remove(word)
2 if isValidWord(word) is false then throw exception end if
3 node = findNode(word)
4 if node != null and node.isEndOfWord and word equals node.fullWord then
5     node.isEndOfWord = false
6     node.fullWord = null
7     toPatriciaTrie(root)
8 end if
```